

APACHE DORIS 3.0.8

学习笔记

第一章：定位、架构与设计思想

从“Doris 是什么”开始，建立实时分析、FE/BE 架构、性能机制与 3.0 部署选择的完整心智模型。

根据本章讨论重新整理

基准版本：Apache Doris 3.0.8

整理日期：2026-08

目录

阅读方式	4
1.1 一句话定位	4
OLTP 与 OLAP 的工作单位不同	4
列存为什么更适合分析	5
1.2 Doris 为什么会出现	5
矛盾一：MySQL 能保存订单，但大分析会拖累业务库	5
矛盾二：离线数仓能处理海量数据，但反馈太慢	6
矛盾三：用很多系统拼出实时服务，架构会越来越重	6
Doris 对这些问题的设计回应	6
真实起点	6
1.3 Doris 的基本架构	6
FE：入口、目录和总指挥	7
FE 高可用	7
BE：存储引擎与执行引擎	7
数据层级	7
一条查询的执行链路	8
一次写入的可见性链路	8
扩容 FE 与扩容 BE 的意义不同	8
1.4 Doris 为什么快	9
第一层：分区、分桶与索引减少读取	9
第二层：列式存储和压缩降低 I/O	9
第三层：谓词下推与 Runtime Filter 尽早丢弃行	9
第四层：向量化提升单核效率	9
第五层：MPP 与两阶段聚合减少总时长和网络	9
第六层：Nereids 选择更便宜的执行计划	10
第七层：预聚合与物化视图复用结果	10

完整的数据量漏斗	10
为什么用了 Doris 仍可能慢	10
1.5 为什么 Doris 适合实时数仓	10
实时包含四段延迟	11
丰富的导入入口	11
为什么采用微批而非每行事务	11
Unique Key 与 Merge-on-Write 承接 CDC	11
事务、Label 与 Exactly-Once 边界	11
三种表模型映射三类实时数据	12
与 Kafka、Flink 的职责分工	12
1.6 Doris 不是什么	12
兼容 MySQL 协议不等于 MySQL	12
流式导入不等于流式计算	12
查询数据湖不等于替代数据湖	12
快速判断	13
1.7 Doris 3.0 的重要选择	13
存算一体：数据靠近计算	13
存算分离：所有计算共享一份主数据	14
无状态 BE 不等于没有磁盘	14
扩容更快不等于没有预热成本	14
完整对比	14
选择建议	14
三个误区	14
3.0.8 的版本说明	15
本章总结	15
十二个必须记住的结论	15
复习题	15

术语速查	16
参考资料	16

阅读方式

本章不是功能清单，而是从问题出发：为什么传统事务库和离线数仓之间会出现 Doris；为什么 FE、BE 要分工；为什么 Doris 能快；为什么它适合作为实时数仓的查询服务层；以及 3.0 为什么同时保留存算一体和存算分离。

- 先理解边界，再记功能：Doris 的核心是实时 OLAP，而不是业务交易数据库。
- 先理解数据量如何被逐层削减，再理解向量化和 MPP。
- 把“实时”拆成采集、导入、发布可见和查询四段延迟。
- 部署模式不存在绝对优劣，只有资源与业务约束下的选择。

1.1 一句话定位

Apache Doris 是一个面向分析场景的分布式实时数据仓库。

分析型意味着一次处理大量数据；分布式意味着把数据与计算拆给多台机器；实时意味着持续写入、较快可见并能快速查询；数据仓库意味着整合多源业务数据，服务统一分析。

概念	含义	典型问题
分析型 OLAP	擅长扫描、过滤、Join、聚合和多维钻取	最近 30 天各省销售额为什么下降？
分布式	数据分片，查询在多台 BE 与多核上并行	30 亿行如何突破单机 CPU、内存和磁盘上限？
实时	数据持续导入、事务发布后快速参与查询	订单刚支付，多久能出现在经营看板？
数据仓库	整合订单、用户、商品、日志等数据，统一指标和历史分析	高级会员浏览某品类后贡献了多少支付金额？

OLTP 与 OLAP 的工作单位不同

比较角度	MySQL 等 OLTP	Doris 等 OLAP
主要目标	正确处理一笔业务事务	高效分析一批业务数据
单次触达数据	一行或少量行	数百万至数十亿行
典型操作	点查、INSERT、UPDATE	扫描、过滤、Join、GROUP BY
关注重点	事务、并发、单行延迟	扫描吞吐、并行和聚合效率
常见存储倾向	行式	列式

Doris 在现代数据链路中的位置



可以把 MySQL 想象成收银台，把 Kafka/Flink 想象成传送带和加工线，把 Doris 想象成经营分析中心。MySQL 记录 “订单 10001 是否支付成功”，Doris 回答 “今天各省已支付订单金额是多少”。

列存为什么更适合分析

分析 SQL 往往只使用宽表中的少数列。列式存储让 Doris 只读取需要的字段；同一列数据类型和分布相似，又更利于字典编码、游程编码与压缩。列存解决 “数据怎么放”，向量化解决 “数据怎么算”，二者共同构成 Doris 分析执行的基础。

```
SELECT province, SUM(pay_amount)
FROM orders
WHERE order_date = '2026-08-15'
GROUP BY province;
```

这条 SQL 通常只需要读取 order_date、province、pay_amount，其余地址、电话、备注等列无需进入扫描路径。

1.2 Doris 为什么会出现

Doris 出现于一个长期存在的能力缺口：事务数据库拥有最新数据，却不擅长海量分析；Hive/Spark 擅长海量离线处理，却不适合低延迟、高并发、交互式查询。

从能力缺口到 Doris



矛盾一：MySQL 能保存订单，但大分析会拖累业务库

索引适合从大量数据中定位少量记录，却不能消除对大量命中行进行聚合、排序和 Join 的成本。任意维度分析也不可能为每一种字段组合都建立联合索引；索引越多，写入维护和空间成本越高。

只读副本可以分摊不同查询并隔离部分压力，但一条扫描 10 亿行的查询仍可能由单个副本承担。Doris MPP 的关键不同是：同一条大查询也能拆到多个 BE 并行执行。

矛盾二：离线数仓能处理海量数据，但反馈太慢

传统 T+1 链路往往在夜间抽取、凌晨计算、第二天出报表。它适合复杂 ETL 和全量历史处理，却难以支撑“发现异常 - 换维度 - 继续下钻”的交互式分析节奏。

矛盾三：用很多系统拼出实时服务，架构会越来越重

当企业同时引入 Kafka、Flink、HBase、Elasticsearch、Hive 和结果库后，同一份指标可能在多个系统重复保存和计算。同步链路越长，一致性、运维和排障成本越高。市场需要一个能够统一承接实时明细、主键状态、聚合分析和标准 SQL 服务的引擎。

Doris 对这些问题的设计回应

业务问题	Doris 的回应	背后的思想
大范围扫描慢	列存、分区裁剪、排序、跳过索引	先少读数据
单机资源有限	Tablet 分片、MPP 多机并行	横向扩展
逐行 CPU 开销高	列式 Block、向量化、SIMD	批量计算
复杂 SQL 计划不稳定	Nereids RBO/CBO	基于规则和代价选计划
固定报表重复计算	Aggregate Key、Rollup、物化视图	提前计算、复用结果
最新数据无法分析	Stream Load、Routine Load、Flink CDC、Unique Key	持续导入并更新
系统使用门槛高	MySQL 协议、标准 SQL、FE/BE 架构	简化接入与运维

真实起点

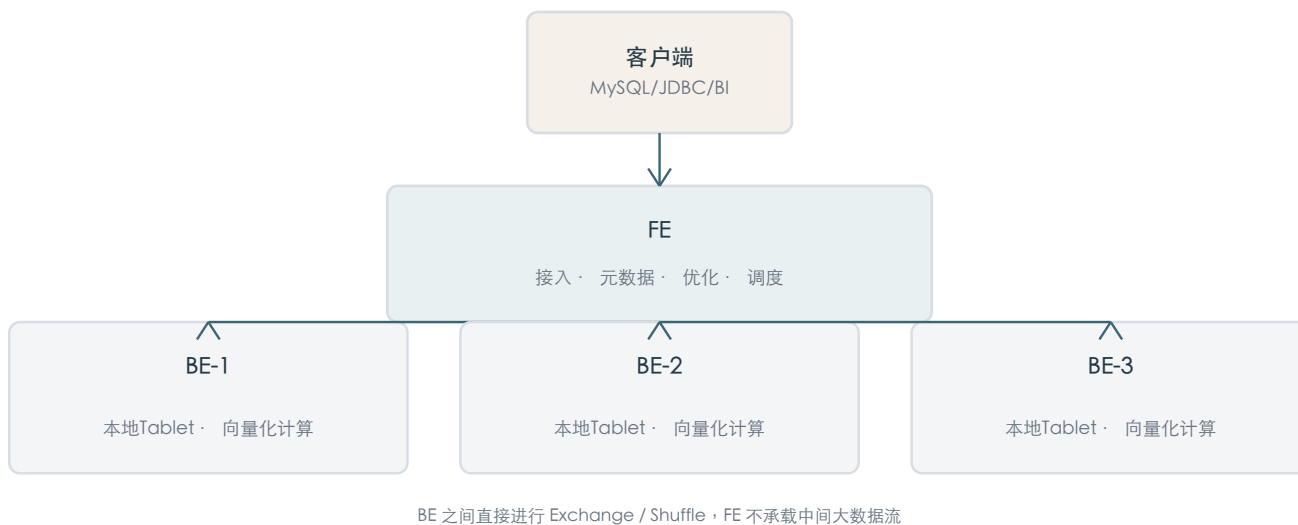
Doris 前身 Palo 最初服务于百度广告报表。广告业务天然需要持续接收曝光、点击和消费数据，并按广告主、计划、地域、设备等大量维度交互查询。这种“最新数据 + 多维聚合 + 高并发报表”的组合，正是实时 OLAP 的典型需求。

Doris 不是因为“数据量大”这一项因素出现，而是因为数据量大、更新快、查询要求快、查询维度不固定同时发生。

1.3 Doris 的基本架构

经典存算一体模式只有两类核心进程：FE 负责思考、管理和协调；BE 负责存储、计算和执行。它本质上是控制面与数据面的分离。

存算一体基本架构



FE：入口、目录和总指挥

- 连接入口：MySQL 协议、认证、权限、Session 和结果返回。
- 元数据：数据库、表、分区、Tablet、副本位置、用户权限、节点状态。
- SQL 规划：语法解析、语义绑定、RBO、CBO、分布式物理计划。
- 查询协调：选择 BE、下发 PlanFragment、跟踪状态并返回结果。
- 集群管理：心跳、负载均衡、副本修复、扩缩容和导入任务。

FE 不承载主要数据扫描。BE 之间直接 Exchange/Shuffle，避免所有中间数据绕经 FE 形成网络瓶颈。

FE 高可用

Follower 组成可选举组，其中一个 Follower 被选为 Master；Master 负责元数据写入，元数据日志通过 BDB JE 复制并经多数派确认。Observer 同步完整元数据并扩展查询接入能力，但不参与 Master 选举。

BE：存储引擎与执行引擎

- 存储：Tablet、Rowset、Segment、列式编码、压缩和索引。
- 执行：Scan、Filter、Join、Aggregate、Sort、向量化与 Pipeline。
- 导入：解析、转换、MemTable、Flush、事务写入和版本发布。
- 后台任务：Compaction、副本管理、Schema Change 和索引构建。

数据层级

理解 BE 时要建立一条固定层级：

从逻辑表到物理文件



层级	作用
Partition	按日期或业务范围管理数据，支持分区裁剪和生命周期管理
Tablet	分区经过分桶后的水平数据分片，是调度、迁移和副本管理的关键单位
Replica	经典模式中 Tablet 在不同 BE 上的物理副本，用于容灾和可用性
Rowset	一次导入或 Compaction 形成的数据版本集合
Segment	实际承载列数据、Page 和索引的文件

一条查询的执行链路

- 1. 客户端向 FE 提交 SQL ； FE 完成鉴权、解析和字段绑定。
- 2. FE 根据元数据进行分区、列和 Tablet 裁剪，并由 Nereids 生成物理计划。
- 3. 完整 Plan 被 Exchange 边界拆成多个 PlanFragment，分别下发到 BE。
- 4. BE 把 Fragment 继续拆成 Pipeline 和多个 PipelineTask，由线程池调度。
- 5. 各 BE 扫描本地 Tablet，进行过滤、局部 Join 或局部聚合。
- 6. 中间结果在 BE 之间直接 Shuffle；最终结果经 FE 返回客户端。

SQL
-> Plan
 -> PlanFragment
 -> Pipeline
 -> PipelineTask
 -> Thread Pool

一次写入的可见性链路

导入事务的核心状态



“文件已写入”和“数据可被一致读取”是两个阶段。多个 Tablet 完成写入后，事务先提交，再发布版本；到达 VISIBLE 后，新查询才读取新快照。查询开始时获取稳定可见版本，执行期间不会突然混入半批新数据。

扩容 FE 与扩容 BE 的意义不同

扩容对象	主要增加	通常不直接增加
FE	连接、SQL 规划、元数据读取能力与控制面高可用	底层数据扫描吞吐
BE	CPU、内存、磁盘、扫描与执行并发	控制面多数派能力

1.4 Doris 为什么快

Doris 的速度不是某一个“黑科技”的结果，而是查询各阶段共同减少成本。最重要的顺序是：少读、少算、批量算、并行算、提前算。

查询耗时可以粗略看成：排队与规划 + 数据读取 + CPU 计算 + 网络 Shuffle + 汇总。性能优化的第一原则不是更快处理全部数据，而是让绝大部分数据根本不进入计算流程。

第一层：分区、分桶与索引减少读取

分区裁剪按日期或业务范围排除整块数据；Bucket/Tablet 裁剪依据 Hash 分桶列的等值条件定位分片；排序 Key、Prefix Index、ZoneMap、Bloom Filter 和倒排索引继续在 Segment 与 Page 层跳过数据。

机制	能跳过什么	适合条件
Partition Pruning	无关分区	WHERE 中包含可推导的分区列范围
Bucket/Tablet Pruning	无关 Tablet	Hash 分桶列等值条件
Prefix Index	排序 Key 之外的数据范围	过滤命中 Key 前导列
ZoneMap	Min/Max 不可能命中的 Segment 或 Page	范围集中、有良好聚集度
Bloom Filter	目标值一定不存在的数据块	高基数等值查询
Inverted Index	非匹配行	文本、等值或范围检索

Doris 没有为每列自动建立昂贵 B+Tree，而是默认维护轻量跳过索引，并让用户按查询模式选择二级索引。这是在读性能、写放大、存储空间与 Compaction 成本之间的折中。

第二层：列式存储和压缩降低 I/O

宽表查询只读所需列；同列相似数据通过字典、游程等编码后更容易压缩。读取少量压缩数据并在内存快速解压，通常比从磁盘读取大量原始字节更划算。对 SELECT * 的宽行点查，列存优势会下降，因此 Doris 提供专门的行列混存与点查路径。

第三层：谓词下推与 Runtime Filter 尽早丢弃行

普通 WHERE 条件被下推到 Scan；Join Runtime Filter 则根据运行时小表键值动态生成过滤器，并下推到大表扫描。例如先筛出 VIP 客户 ID，再在读取 orders 时排除绝大多数非 VIP 订单，可以同时减少 I/O、Join Probe 和网络传输。

第四层：向量化提升单核效率

向量化不是逐行 next()，而是让算子消费和产生列式 Block。批量连续内存、紧凑循环和较少虚函数调用更利于 CPU 缓存与 SIMD。列存负责让同一列连续到达内存，向量化负责成批过滤、表达式计算和聚合。

概念	解决的问题	不要混淆
向量化	一批数据怎样高效使用 CPU	决定处理单位和内存布局
Pipeline	算子和任务怎样调度、并行与等待	决定执行并发和线程利用
MPP	一条查询怎样跨多台 BE 并行	决定多机分布式执行

第五层：MPP 与两阶段聚合减少总时长和网络

各 BE 先在本地图扫描自己的 Tablet 并做 Partial Aggregate，再按分组键 Exchange，最后做 Final Aggregate。能在数据所在节点完成的计算尽量先完成，网络中传输局部结果，而不是全部明细。

第六层：Nereids 选择更便宜的执行计划

RBO 负责列裁剪、谓词下推、分区裁剪、常量折叠等确定性改写；CBO 根据行数、基数、过滤选择率和网络成本决定 Join 顺序、Broadcast 或 Shuffle。统计信息不准确时，CBO 也可能广播大表或选择低效顺序。

第七层：预聚合与物化视图复用结果

Aggregate Key 把同 Key 数据按 SUM、MAX、REPLACE 等语义合并；同步或异步物化视图保存预计算结果，优化器可透明改写查询。它们把成本从每次查询转移到写入或刷新阶段，适合重复度高的报表，但会增加写入、刷新和存储成本。

完整的数据量漏斗

查询越早减少数据，后续每个算子收益越大



为什么用了 Doris 仍可能慢

- 分区裁剪失败，或者 Key 顺序与高频过滤条件不匹配。
- Tablet 太少限制并行，太多则增加元数据、调度与 Compaction 开销。
- 分桶键倾斜，最慢任务决定整条查询完成时间。
- 大表 Join 需要传输大量 Shuffle 数据，或者统计信息误导优化器。
- 高频小批导入制造大量 Rowset 和版本，Compaction 追不上。
- 异步物化视图过细或没有命中，刷新成本大于查询收益。

1.5 为什么 Doris 适合实时数仓

实时数仓不是“数据能够流进来”就完成，而是数据要持续进入、正确更新、一致可见，并且能立即被快速分析。Doris 把导入、主键更新、事务发布、列式存储和查询服务放在同一引擎内。

实时数仓的端到端链路



实时包含四段延迟

阶段	要解决的问题
采集实时	业务变化多久被 CDC 捕获
写入实时	Kafka/Flink/应用数据多久进入 Doris
可见实时	导入何时从 COMMITTED 发布到 VISIBLE
查询实时	新数据可见后，SQL 多久返回

端到端延迟约等于 CDC、消息队列、Flink、Doris 导入、版本发布与查询延迟之和。“秒级实时”应明确是单段平均值，还是从源库变化到用户看到结果的端到端 P99。

丰富的导入入口

来源	推荐入口	特点
HTTP / 应用批次	Stream Load	同步返回每批事务结果
Kafka	Routine Load	持续微批消费并记录 Offset
Flink / CDC	Flink Doris Connector	与 Checkpoint、2PC 组合
JDBC 高频小写入	INSERT + Group Commit	服务端合并小事务
S3 / HDFS 历史补数	Broker Load / INSERT SELECT	异步大批量导入

为什么采用微批而非每行事务

每行一个分布式事务会带来重复规划、版本发布、大量小 Rowset 和 Compaction 压力。微批在较短时间窗口内合并事件，用很小的新鲜度延迟换取高吞吐与可持续运行。Group Commit 的 sync_mode 等待合并事务完成，async_mode 可先写 WAL 后返回，再异步发布。

Unique Key 与 Merge-on-Write 承接 CDC

订单状态、用户画像和维度表并非只追加。Unique Key 将相同主键的变更表达为 UPSERT；Merge-on-Write 在写入阶段查找旧键并通过 Delete Bitmap 标记旧版本，查询直接读取最新有效结果，Compaction 再清理物理旧数据。它用更高写入成本换取更稳定的读性能。

```
CREATE TABLE dwd_order (
  order_id BIGINT,
  update_time DATETIME,
  status VARCHAR(20),
  pay_amount DECIMAL(18,2)
)
UNIQUE KEY(order_id)
DISTRIBUTED BY HASH(order_id) BUCKETS 16
PROPERTIES (
  "enable_unique_key_merge_on_write" = "true"
);
```

事务、Label 与 Exactly-Once 边界

Label 为一批导入提供幂等身份，网络超时后的重试应复用同一 Label。Routine Load 把 Kafka 消费进度与导入事务协调；Flink Connector 可用 Stream Load 2PC 将 Doris 提交与 Flink Checkpoint 对齐。端到端 Exactly-Once 仍依赖上游 Source、Checkpoint、Connector、Label 和主键顺序共同正确配置。

三种表模型映射三类实时数据

数据形态	建议模型	典型内容
原始事件	Duplicate Key	点击、日志、行为、流水明细
实体最新状态	Unique Key MOW	订单、用户标签、商品、CDC 镜像
实时聚合指标	Aggregate Key / 物化视图	分钟 PV、广告点击、省份销售额

与 Kafka、Flink 的职责分工

Kafka 负责传输和缓冲；Flink 负责事件时间、窗口、状态和复杂流式加工；Doris 负责长期存储、主键最新状态、Join/聚合和面向用户的 SQL 服务。简单 Kafka 数据可直接 Routine Load；需要复杂窗口、乱序处理和多流 Join 时，采用 Kafka - Flink - Doris。

实时写入解决数据新鲜度，高性能查询解决使用新鲜数据的速度。两者必须同时成立，Doris 才能成为实时数仓服务层。

1.6 Doris 不是什么

Doris 可以覆盖相邻系统的一部分能力，但它的核心职责仍然是实时分析。明确边界能避免“什么都放进 Doris”带来的架构错误。

Doris 不是什么	相邻系统更擅长	Doris 合理角色
不是核心 OLTP 交易库	MySQL/PostgreSQL：复杂小事务、业务约束、频繁单行修改	订单状态同步后的分析、分析型点查
不是消息队列	Kafka：缓冲、回放、多消费者、削峰	消费消息并持久化为可查询数据
不是复杂流计算引擎	Flink：事件时间、Watermark、CEP、长状态	接收流计算结果并提供长期查询
不是数据湖本身	Iceberg/Hive：开放格式、低成本历史存储、多引擎共享	湖上查询加速或热点数据服务
不是纯搜索引擎	Elasticsearch：相关性、分词与搜索应用生态	检索与 SQL Join/聚合结合
不是完整治理平台	编排、血缘、质量、审批、指标管理系统	存储、计算和查询服务
不是自动变快的魔法	正确建模、资源与运维不可替代	提供性能能力，效果取决于设计

兼容 MySQL 协议不等于 MySQL

Doris 兼容 MySQL 客户端、JDBC/ODBC 与部分语法，是为了降低接入成本；它的列式存储、分布式事务、批量导入、Tablet 与 MPP 执行都与 MySQL 核心设计不同。Doris 可以做高并发分析型点查，但不应因此承担资金转账、下单扣库存等核心业务事务。

流式导入不等于流式计算

Routine Load 的重点是持续消费、轻量转换和事务写入。需要事件时间窗口、乱序处理、CEP 与复杂状态时，应先用 Flink 计算，再把结果写入 Doris。

查询数据湖不等于替代数据湖

大量冷数据、原始历史数据和开放格式共享仍适合对象存储与湖表格式；Doris 更适合保存热点、服务型数据或作为湖上高性能查询层。

快速判断

需求	适合程度
实时经营看板、多维即席分析、用户画像	很适合
订单 CDC 后的最新状态查询	很适合
日志检索并进行 SQL 聚合	适合
湖仓数据查询加速	适合
银行转账核心库、电商下单主库	不适合
Kafka 消息缓冲和回放	不适合
复杂事件时间流计算	不适合
只做极低成本冷数据归档	通常不是首选

1.7 Doris 3.0 的重要选择

Doris 3.0 开始支持两种部署模式：存算一体 Local Mode 与存算分离 Cloud Mode。它们不是新旧淘汰关系，而是把性能、弹性、成本和运维复杂度放在不同位置。

Doris 3.0 的两种部署模式



存算一体：数据靠近计算

BE 同时拥有 CPU、内存、本地磁盘、Tablet 副本和查询执行能力。查询直接读取本地 SSD，路径短且延迟可预测；新增 BE 同时增加存储与计算，但需要迁移和均衡 Tablet。数据可靠性主要通过 Doris 管理的多副本实现。

- 优势：FE + BE 架构简洁、本地访问快、性能稳定、排障直接。
- 代价：计算与存储绑定；本地多副本 SSD 成本较高；扩缩容伴随数据迁移。
- 适合：本地机房、负载稳定、重视低延迟和简单运维的团队。

存算分离：所有计算共享一份主数据

主数据存入 S3、HDFS、OSS、MinIO 等共享存储；FE 继续负责 SQL 接入与规划；Meta Service 管理导入事务、Tablet、Rowset 和计算资源元数据，并使用 FoundationDB 持久化关键状态；无状态 BE 组成计算组并使用本地 SSD 作为 File Cache。

- 优势：计算与存储独立扩缩；多计算组可物理隔离工作负载；对象存储适合冷热分层。
- 代价：增加 Meta Service、FoundationDB 与共享存储依赖；缓存未命中会带来网络读取和首查抖动。
- 适合：公有云/私有云、Kubernetes、负载峰谷明显、多团队共享数据且需要资源隔离。

无状态 BE 不等于没有磁盘

存算分离 BE 的本地盘不再保存唯一主数据，而主要作为高性能缓存。缓存命中时走本地 SSD；未命中时从共享存储下载并填充缓存。因此缓存容量、热点比例、共享存储吞吐与网络延迟是性能关键。

扩容更快不等于没有预热成本

存算一体新增 BE 要迁移 Tablet；存算分离新增 BE 可以直接访问共享数据，但新节点缓存为空，首次查询仍可能较慢。它消除了主数据搬迁前置条件，却没有消除缓存预热。

完整对比

维度	存算一体	存算分离
主数据位置	BE 本地磁盘	共享存储
BE 角色	持久存储 + 计算	计算 + 本地缓存
元数据组件	FE + BE 本地元数据	FE + Meta Service + FoundationDB
读取路径	本地磁盘	缓存命中本地读，未命中远端读
计算扩容	同时增加存储	独立增加计算组或 BE
扩容成本	Tablet 迁移与均衡	主数据无需迁移，需缓存预热
工作负载隔离	主要是逻辑资源隔离	多计算组物理隔离
存储成本	本地多副本 SSD	低成本共享存储 + 热数据缓存
性能特征	低延迟、稳定、可预测	受缓存命中与远端存储影响
运维复杂度	较低	较高

选择建议

- 优先存算一体：负载稳定、有本地 SSD、缺少可靠共享存储、追求低延迟和简单运维。

优先评估存算分离：计算/存储要独立扩缩、多团队需要物理隔离、冷数据多、运行在云和 Kubernetes、且有平台团队维护。

三个误区

1. “存算分离一定更先进、更快”是错误的。它优化弹性、隔离和成本；存算一体优化本地路径、稳定性和简单。
2. “存算分离 BE 不需要磁盘”是错误的。File Cache 对热点查询延迟极其重要。
3. “部署后可以随时切换模式”是错误的。Local Mode 与 Cloud Mode 不是运行时开关，转换通常需要新集群和数据迁移。

3.0.8 的版本说明

3.0.8 是 3.0 分支补丁版本，核心架构沿用 3.0。该版本继续改进导入内存不足时的 Flush 策略，并在缓存空间允许时，让存算分离 Base Compaction 生成的 Rowset 进入 File Cache。架构模式仍应在部署前确定。

建议学习顺序：先掌握存算一体的 Table - Partition - Tablet - Replica - Rowset - Segment，再理解存算分离怎样把主数据移到共享存储、把数据层元数据抽到 Meta Service，并把 BE 变成带缓存的计算节点。

本章总结

十二个必须记住的结论

1. Doris 是实时 OLAP 数据库，不是传统 OLTP 数据库。
2. MySQL 处理一笔业务，Doris 分析一批业务数据。
3. Doris 出现在事务库做不动大分析、离线数仓来不及回答实时问题的夹缝中。
4. FE 保存目录、生成计划并协调任务；BE 保存和处理数据。
5. BE 之间直接 Exchange，FE 不应成为中间数据通道。
6. Doris 快的第一原则是少读，其次才是更快计算。
7. 列存、向量化、Pipeline 和 MPP 分别解决不同层次的问题。
8. RBO 做确定性改写，CBO 依赖统计信息选择更低成本计划。
9. 实时数仓要同时满足持续导入、正确更新、一致可见和快速查询。
10. Unique Key Merge-on-Write 把主键合并成本放在写入阶段，换取稳定读取。
11. Doris 不替代 MySQL、Kafka、Flink 和数据湖，而是在链路中承担分析服务层。
12. 存算一体以本地数据路径换性能与简单；存算分离以共享主数据换弹性、隔离和成本优势。

复习题

1. 为什么给 MySQL 增加多个只读副本，仍然不等于得到 Doris？
2. FE 为什么不能承担 BE 之间的所有 Shuffle 数据？
3. Partition、Tablet、Replica、Rowset、Segment 分别是什么？
4. 列式存储、向量化、Pipeline 和 MPP 的区别是什么？
5. 为什么 Runtime Filter 能同时减少 I/O、Join 和网络成本？
6. COMMITTED 与 VISIBLE 的区别是什么？
7. 为什么实时数仓通常采用微批而不是逐行事务？
8. 什么时候应把复杂处理放在 Flink，而不是 Doris Routine Load？
9. 为什么存算分离扩容更快，却仍可能出现首查抖动？
10. 你的业务更适合 Local Mode 还是 Cloud Mode？决定因素是什么？

术语速查

术语	一句话解释
FE	SQL 接入、元数据、优化器和查询协调节点
BE	存储数据并执行扫描、Join、聚合的节点
Tablet	分区经分桶后的水平数据分片
Rowset	一次导入或 Compaction 形成的数据版本集合
Segment	实际承载列数据、Page 与索引的文件
PlanFragment	FE 下发给 BE 的分布式执行片段
PipelineTask	BE 线程池实际调度的执行任务
Runtime Filter	执行中根据一侧数据动态生成并下推的过滤器
Merge-on-Write	写入时处理主键旧版本，查询直接读取最新有效版本
Meta Service	存算分离中的数据层元数据与事务服务
File Cache	存算分离 BE 上的本地高速数据缓存

参考资料

以下资料均为 Apache Doris 官方文档或官方 Release Notes。本章以 3.0.8 为基准，主体机制参考 3.0 分支；部分长期维护页面使用 3.x 路径。

- [1] Introduction to Apache Doris
<https://doris.apache.org/docs/3.0/gettingStarted/what-is-apache-doris/>
- [2] Release 3.0.8
<https://doris.apache.org/releases/v3.0/release-3.0.8/>
- [3] Pipeline Execution Engine
<https://doris.apache.org/docs/3.0/query-acceleration/optimization-technology-principle/pipeline-execution-engine>
- [4] Query Optimizers
<https://doris.apache.org/docs/3.x/query-acceleration/optimization-technology-principle/query-optimizer/>
- [5] Runtime Filter
<https://doris.apache.org/docs/3.0/query-acceleration/optimization-technology-principle/runtime-filter>
- [6] Loading Overview
<https://doris.apache.org/docs/3.x/data-operate/import/load-manual>
- [7] Transaction
<https://doris.apache.org/docs/3.x/data-operate/transaction/>
- [8] Group Commit
<https://doris.apache.org/docs/3.x/data-operate/import/group-commit-manual/>
- [9] Routine Load
<https://doris.apache.org/docs/3.x/data-operate/import/import-way/routine-load-manual/>
- [10] Unique Key Table
<https://doris.apache.org/docs/3.x/table-design/data-model/unique/>
- [11] Materialized View Overview
<https://doris.apache.org/docs/3.x/query-acceleration/materialized-view/overview/>
- [12] Compute-Storage Decoupled Overview
<https://doris.apache.org/docs/3.x/compute-storage-decoupled/overview/>
- [13] Compute-Storage Decoupled Upgrade
<https://doris.apache.org/docs/3.x/compute-storage-decoupled/upgrade/>